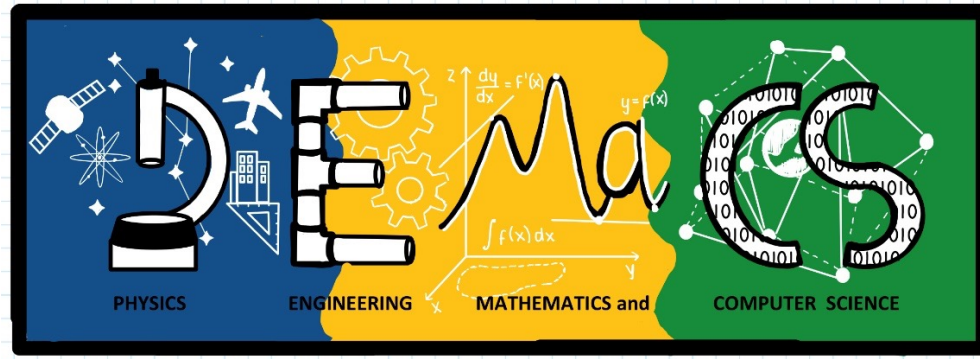




*Division of Physics, Engineering, Mathematics,
and Computer Science*

CSCI-121 Elements of Computer Programming II

Staying Left of the Line

Marwan Rasamny

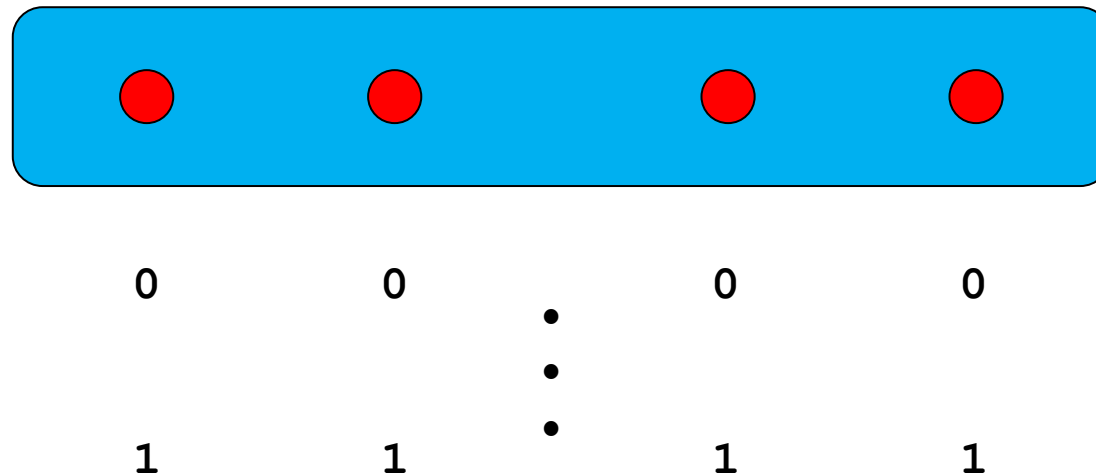
Learning Objectives

- At the end of this session, a student will be able to
 - Implement the line walking algorithm.
 - Use registration to register the line walking algorithm on the mBot2.
 - Implement the command on the Java end of the framework.



Quad RGB Line Sensor

- The Quad RGB sensor has 4 sensor that returns line status information as an integer between 0 and 15.
 - When used to distinguish between black/white, the quad RGB returns a 0 or 1 for each of its on-board sensors.



Binary Numbers

- A bit can hold two values, 0 or 1.
- Two bits can hold four values (i.e. 2 for each bit)

0	0	0
0	1	1
1	0	2
1	1	3

$$11_2 = 1 \times 2^1 + 1 \times 2^0 = 3_{10}$$

➤ What does $1011_2 = ?$

- $1011_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$
- $1011_2 = 1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1$
- $0010_2 = 8 + 0 + 2 + 1$
- $1011_2 = 11_{10}$



Line Sensing



$$0001_2 = 0 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$0001_2 = 0 \times 8 + 0 \times 4 + 0 \times 2 + 1 \times 1$$

$$0010_2 = 0 + 0 + 0 + 1$$

$$0001_2 = \mathbf{1}_{10}$$

`quad_rgb_sensor.get_line_sta()` returns a **1**



Line Sensing, Cont'd

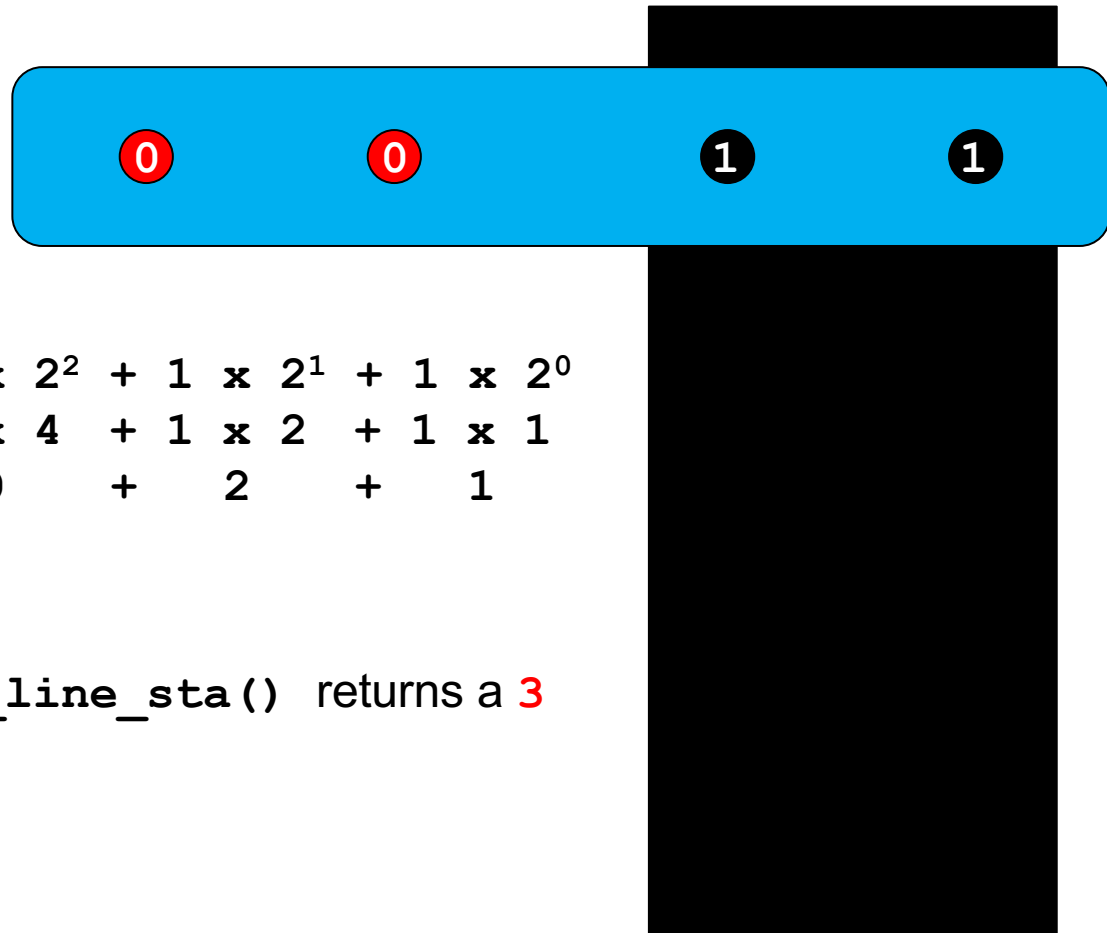


$$\begin{aligned}0010_2 &= 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \\0010_2 &= 0 \times 8 + 0 \times 4 + 1 \times 2 + 0 \times 1 \\0010_2 &= 0 + 0 + 2 + 0 \\0010_2 &= 2_{10}\end{aligned}$$

`quad_rgb_sensor.get_line_sta()` returns a **2**



Line Sensing, Cont'd

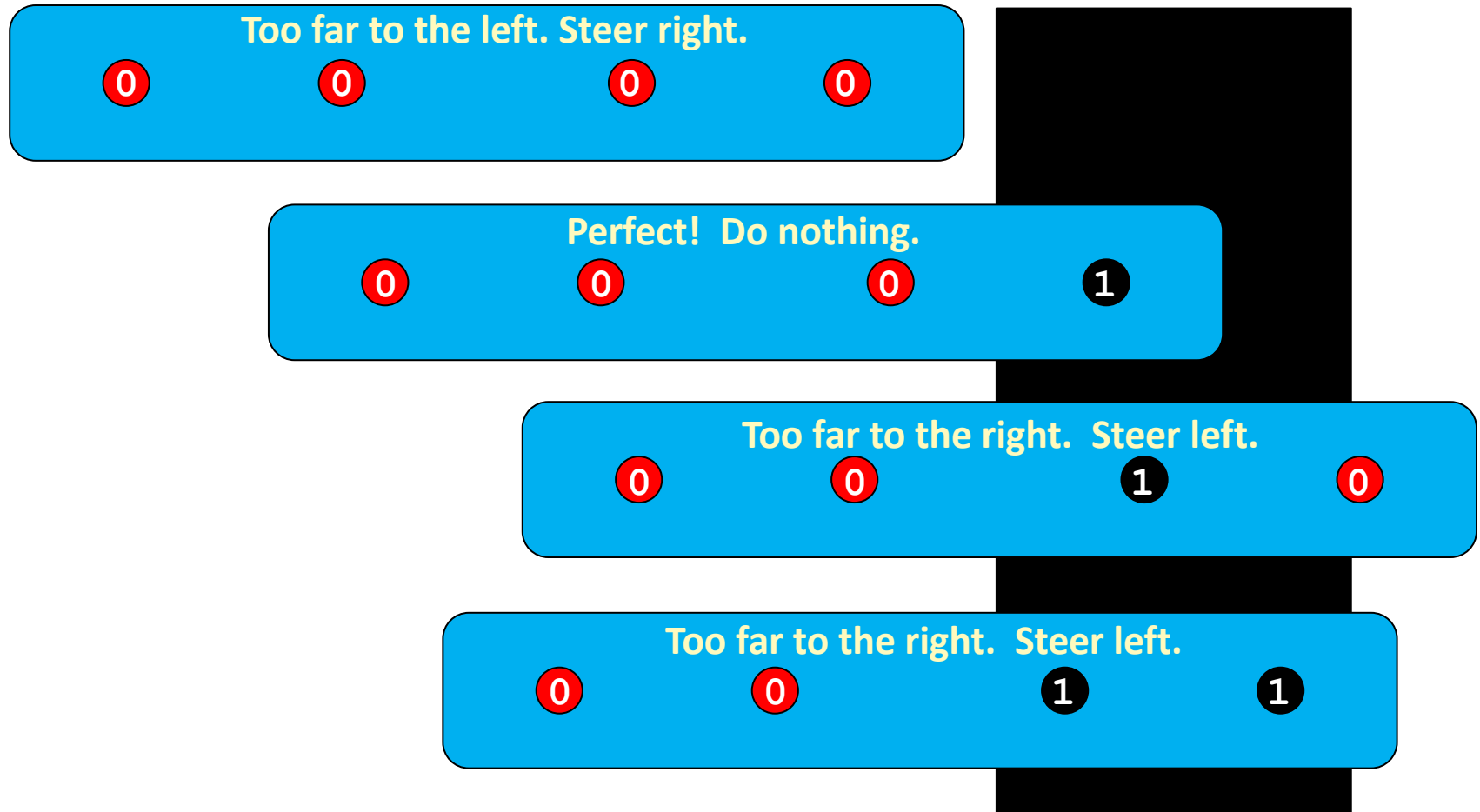


$$\begin{aligned}0011_2 &= 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\0011_2 &= 0 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 \\0010_2 &= 0 + 0 + 2 + 1 \\0011_2 &= 3_{10}\end{aligned}$$

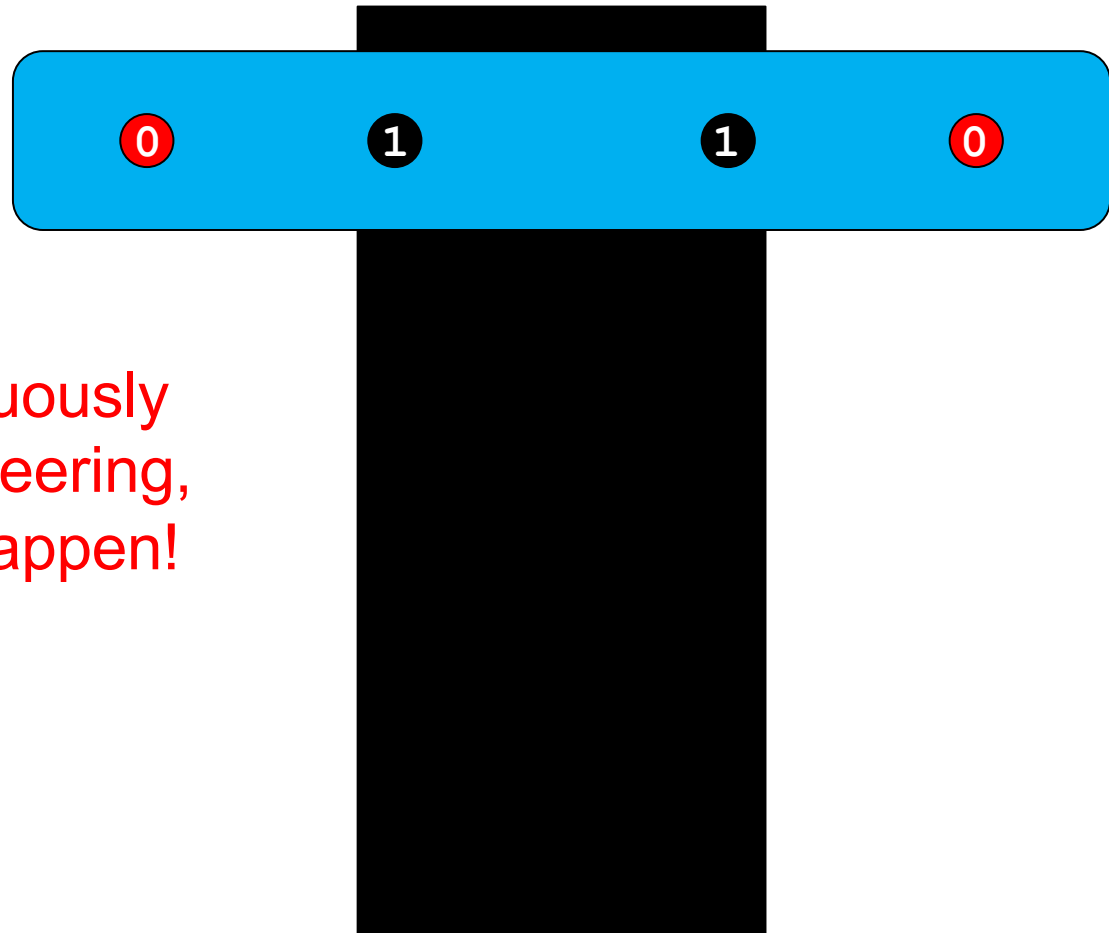
`quad_rgb_sensor.get_line_sta()` returns a **3**



What to do?



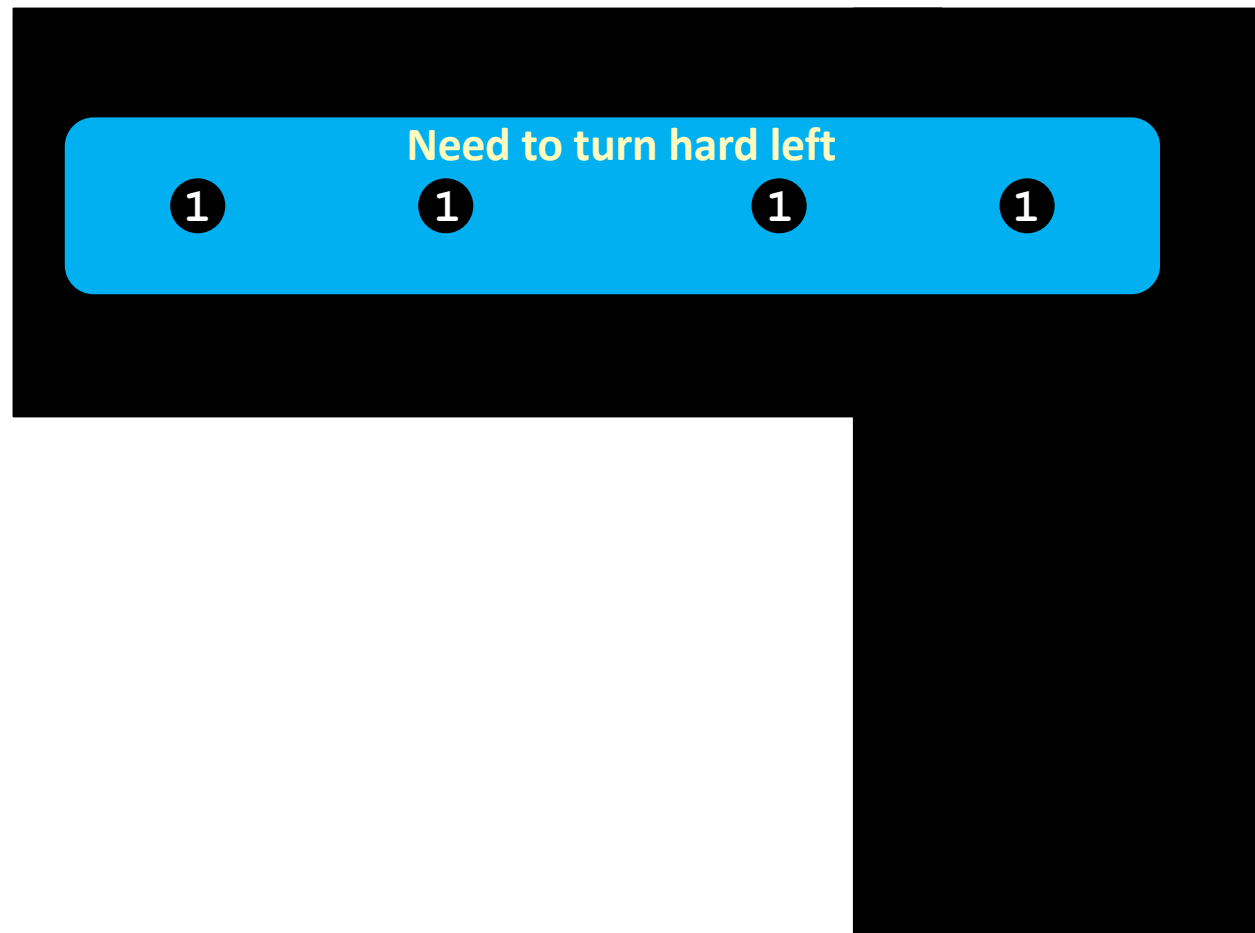
Avoid This Scenario



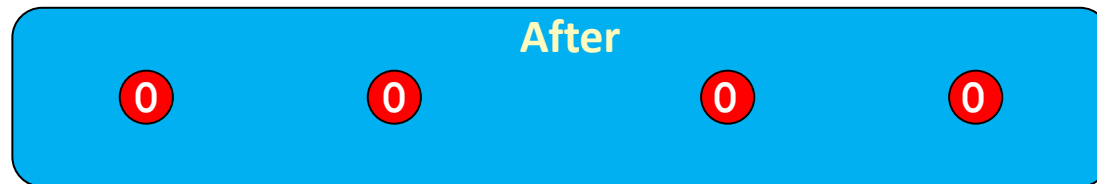
If you are continuously
monitoring and steering,
this should not happen!



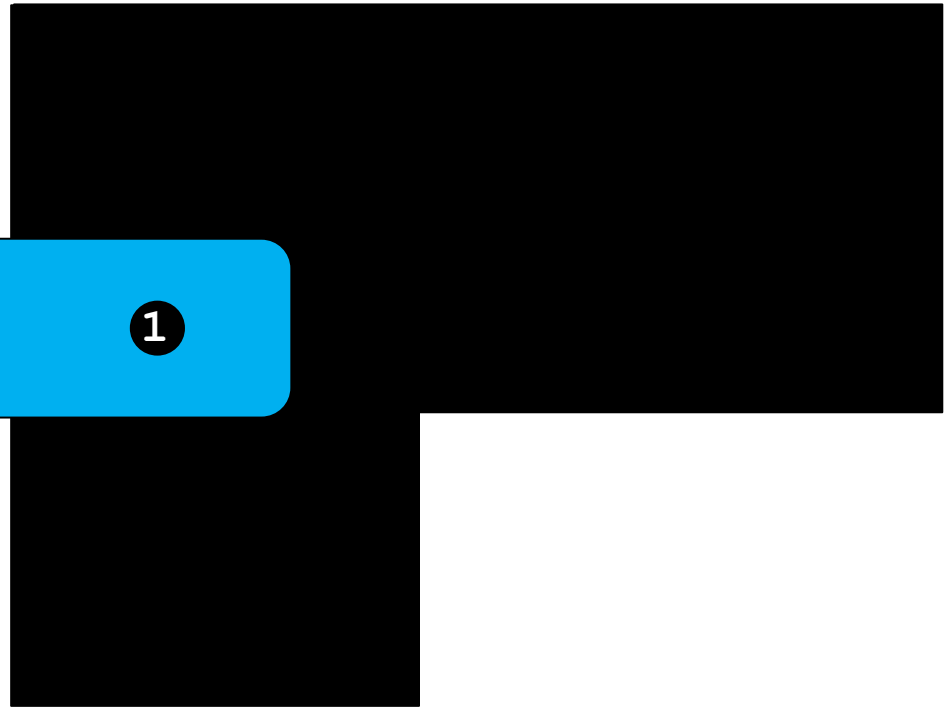
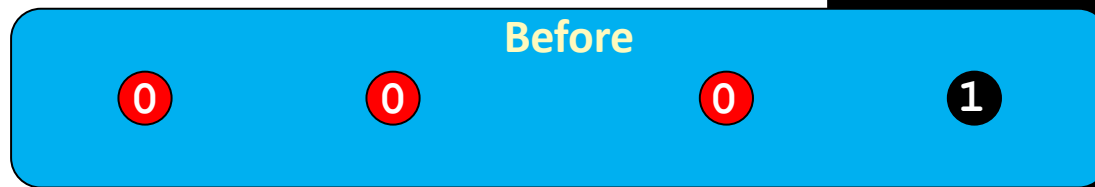
What About Left Angle Turns



What About Right Angle Turns



This case is already addressed



Steering

- Remember that we cannot use the turn functions provided by the API.
 - We need to change the speed or power on each motor to turn and allow the robot to continue moving forward or backward.
 - We use the
 - `mbot2.drive_speed(em1_speed, em2_speed)`
 - `em1_speed` must be **positive** for wheel to move forward
 - `em2_speed` must be **negative** for wheel to move forward
- You should use a scheduled behavior with a low priority to walk the left line.



Steering – Proportional Control

1. Read line status, call it `line`
2. Set `kp` to 0.4
3. Set `base_speed` = 30
4. if `line` == 0, then set error
so that the mbot2 steers right
5. else if `line` > 1 and `line` < 4, then set error
to steer mbot slightly left
6. else if `line` < 7, then set error to a steer
mbot left slightly harder than in step 4
7. else set error to steer mbot hard left.
8. Set `correction` = `error` * `kp`
9. `em1_speed` = `base_speed` + `correction`
10. `em1_speed` = `min(max(em1_speed, -50), 50)`
11. `em2_speed` = `-base_speed` + `correction`
12. `em2_speed` = `min(max(em2_speed, -50), 50)`
13. `mbot2.drive_speed(em1_speed, em2_speed)`

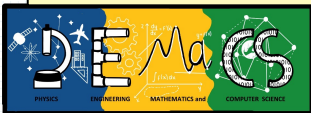


Registering Line Following Behavior

- Add a line following handler and line following behavior at the end of the **server_mbot.py** file.

```
def follow_line_behavior():
    if arbiter.acquire("line", " FOLLOW_LINE ", 10, blocking=False):
        return
    try:
        status = mbuild.quad_rgb_sensor.get_line_sta() // line 1
    finally:
        arbiter.release("line", " FOLLOW_LINE ")
    if not arbiter.acquire("motors", " FOLLOW_LINE ", 10, blocking=False):
        return
    try:
        // implement line 2 - 13
    finally:
        arbiter.release("motors", " FOLLOW_LINE ")

@register_command("FOLLOW_LINE")
def handle_follow_line(payload):
    scheduler.start_behavior("FOLLOW_LINE", follow_line_behavior)
    return ok_response("Following Line")
```



Creating Command In MBot2

- Add a method in the MBot2 class.

```
/**  
 * Commands the robot to walk left of the line.  
 */  
public void followLine() {  
    execute("FOLLOW_LINE",null);  
}
```

